

**Universidad de las Américas Puebla (UDLAP)**

**Licenciatura en Ciencia de Datos**

# Actividad 12: Implementación de monitores

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 20 de octubre de 2026

# Índice general

**1. Productor–consumidor con monitor** **1**

1.1. Entradas y procesamiento . . . . . 1

1.2. Prueba de escritorio . . . . . 1

1.3. Resultados . . . . . 2

**2. Filósofos comensales con monitor** **3**

2.1. Entradas y procesamiento . . . . . 3

2.2. Prueba de escritorio . . . . . 4

2.3. Resultados . . . . . 4

**3. Resultados comparados y análisis** **6**

# Índice de figuras

1.1. Código y salida real del productor–consumidor con monitor. 2

2.1. Código y salida real de los filósofos con monitor. . . . . 5

# Índice de cuadros

1.1. Operaciones del búfer (capacidad 5). . . . . 1

2.1. Ejemplo de dos filósofos vecinos. . . . . 4

3.1. Resultados del productor–consumidor en diez ejecuciones. . 6
3.2. Resultados de los filósofos en diez ejecuciones. . . . . 6

# 1. Productor–consumidor con monitor

**Instrucción.** Implementar un búfer circular con cinco espacios, dos productores que depositan veinte elementos cada uno y dos consumidores que retiran veinte elementos cada uno. Los métodos `deposit()` y `fetch()` deben ser `synchronized`, usar `wait()` y `notifyAll()`, y verificar las condiciones con `while`.

## 1.1 Entradas y procesamiento

El búfer almacena cinco enteros, con `inBuf`, `outBuf` y `cantidad` inicializados en cero. Cada productor genera valores únicos: del 1 al 20 y del 21 al 40. `deposit()` espera cuando `cantidad==5`; `fetch()` espera cuando `cantidad==0`. Ambos métodos modifican el arreglo bajo el monitor y notifican a los hilos bloqueados.

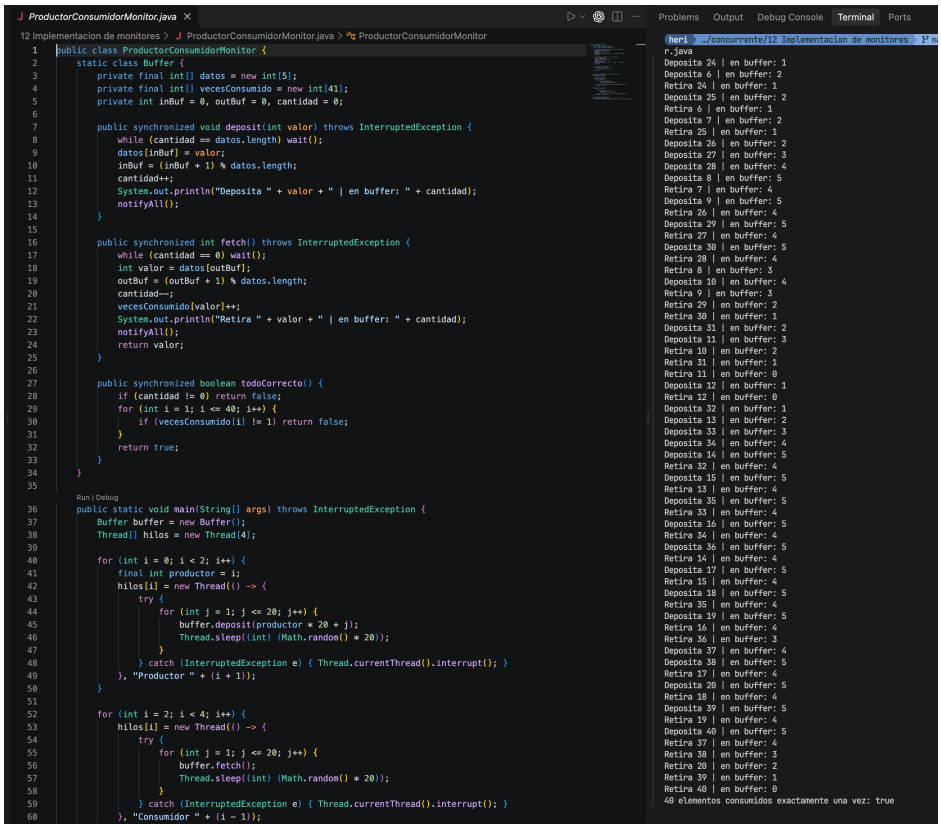
## 1.2 Prueba de escritorio

**Cuadro 1.1:** Operaciones del búfer (capacidad 5).

| Paso | Operación | inBuf | outBuf | Cantidad | Estado |
|---|---|---|---|---|---|
| 0 | Inicial | 0 | 0 | 0 | Vacío |
| 1 | Depositar 1 | 1 | 0 | 1 | [1, –, –, –, –] |
| 2 | Depositar 2 | 2 | 0 | 2 | [1, 2, –, –, –] |
| 3 | Retirar 1 | 2 | 1 | 1 | [–, 2, –, –, –] |
| 4 | Retirar 2 | 2 | 2 | 0 | Vacío; otro retiro esperaría |

## 1.3 Resultados

Se realizaron diez ejecuciones con pausas aleatorias. Las diez finalizaron; en cada una se verificó que los cuarenta elementos se consumieron exactamente una vez.



```java
public class ProductorConsumidorMonitor {
    static class Buffer {
        private final int[] datos = new int[5];
        private final int[] vecesConsumido = new int[41];
        private int inBuf = 0, outBuf = 0, cantidad = 0;

        public synchronized void deposit(int valor) throws InterruptedException {
            while (cantidad == datos.length) wait();
            datos[inBuf] = valor;
            inBuf = (inBuf + 1) % datos.length;
            cantidad++;
            System.out.println("Deposita " + valor + " | en buffer: " + cantidad);
            notifyAll();
        }

        public synchronized int fetch() throws InterruptedException {
            while (cantidad == 0) wait();
            int valor = datos[outBuf];
            outBuf = (outBuf + 1) % datos.length;
            cantidad--;
            vecesConsumido[valor]++;
            System.out.println("Retira " + valor + " | en buffer: " + cantidad);
            notifyAll();
            return valor;
        }

        public synchronized boolean todoCorrecto() {
            if (cantidad != 0) return false;
            for (int i = 1; i <= 40; i++) {
                if (vecesConsumido[i] != 1) return false;
            }
            return true;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Buffer buffer = new Buffer();
        Thread[] hilos = new Thread[4];

        for (int i = 0; i < 2; i++) {
            final int productor = i;
            hilos[i] = new Thread(() -> {
                try {
                    for (int j = 1; j <= 20; j++) {
                        buffer.deposit(productor * 20 + j);
                        Thread.sleep((int) (Math.random() * 20));
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }, "Productor " + (i + 1));
        }

        for (int i = 2; i < 4; i++) {
            hilos[i] = new Thread(() -> {
                try {
                    for (int j = 1; j <= 20; j++) {
                        buffer.fetch();
                        Thread.sleep((int) (Math.random() * 20));
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }, "Consumidor " + (i - 1));
```

```
Deposita 24 | en buffer: 1
Deposita 6 | en buffer: 2
Retira 24 | en buffer: 1
Deposita 25 | en buffer: 2
Retira 6 | en buffer: 1
Deposita 7 | en buffer: 2
Retira 25 | en buffer: 1
Deposita 26 | en buffer: 2
Deposita 27 | en buffer: 3
Deposita 28 | en buffer: 4
Deposita 8 | en buffer: 5
Retira 7 | en buffer: 4
Deposita 9 | en buffer: 5
Retira 26 | en buffer: 4
Deposita 29 | en buffer: 5
Retira 27 | en buffer: 4
Deposita 30 | en buffer: 5
Retira 28 | en buffer: 4
Retira 8 | en buffer: 3
Deposita 10 | en buffer: 4
Retira 9 | en buffer: 3
Retira 29 | en buffer: 2
Retira 30 | en buffer: 1
Deposita 31 | en buffer: 2
Deposita 11 | en buffer: 3
Retira 10 | en buffer: 2
Retira 31 | en buffer: 1
Retira 11 | en buffer: 0
Deposita 12 | en buffer: 1
Retira 12 | en buffer: 0
Deposita 32 | en buffer: 1
Deposita 13 | en buffer: 2
Deposita 33 | en buffer: 3
Deposita 34 | en buffer: 4
Deposita 14 | en buffer: 5
Retira 32 | en buffer: 4
Deposita 15 | en buffer: 5
Retira 13 | en buffer: 4
Deposita 35 | en buffer: 5
Retira 33 | en buffer: 4
Deposita 16 | en buffer: 5
Retira 34 | en buffer: 4
Deposita 36 | en buffer: 5
Retira 14 | en buffer: 4
Deposita 17 | en buffer: 5
Retira 15 | en buffer: 4
Deposita 18 | en buffer: 5
Retira 35 | en buffer: 4
Deposita 19 | en buffer: 5
Retira 16 | en buffer: 4
Retira 36 | en buffer: 3
Deposita 37 | en buffer: 4
Deposita 38 | en buffer: 5
Retira 17 | en buffer: 4
Deposita 20 | en buffer: 5
Retira 18 | en buffer: 4
Deposita 39 | en buffer: 5
Retira 19 | en buffer: 4
Deposita 40 | en buffer: 5
Retira 37 | en buffer: 4
Retira 38 | en buffer: 3
Retira 20 | en buffer: 2
Retira 39 | en buffer: 1
Retira 40 | en buffer: 0
40 elementos consumidos exactamente una vez: true
```

**Figura 1.1:** Código y salida real del productor–consumidor con monitor.

# 2. Filósofos comensales con monitor

**Instrucción.** Implementar cinco hilos filósofos con un monitor, el arreglo de estados PENSANDO, HAMBRIENTO y COMIENDO, un método para solicitar permiso, otro para terminar y `test(i)` para revisar vecinos. Cada filósofo intenta comer veinte veces.

## 2.1 Entradas y procesamiento

El monitor `Mesa` almacena `estado[5]` y `veces[5]`. El método `pedirComer(i)` cambia el estado a HAMBRIENTO y utiliza `test(i)`. Mientras no pueda comer ejecuta `wait()`, que libera el monitor. Al terminar de comer, `terminarComer(i)` cambia el estado a PENSANDO, incrementa el contador del filósofo, revisa a sus vecinos y ejecuta `notifyAll()`.

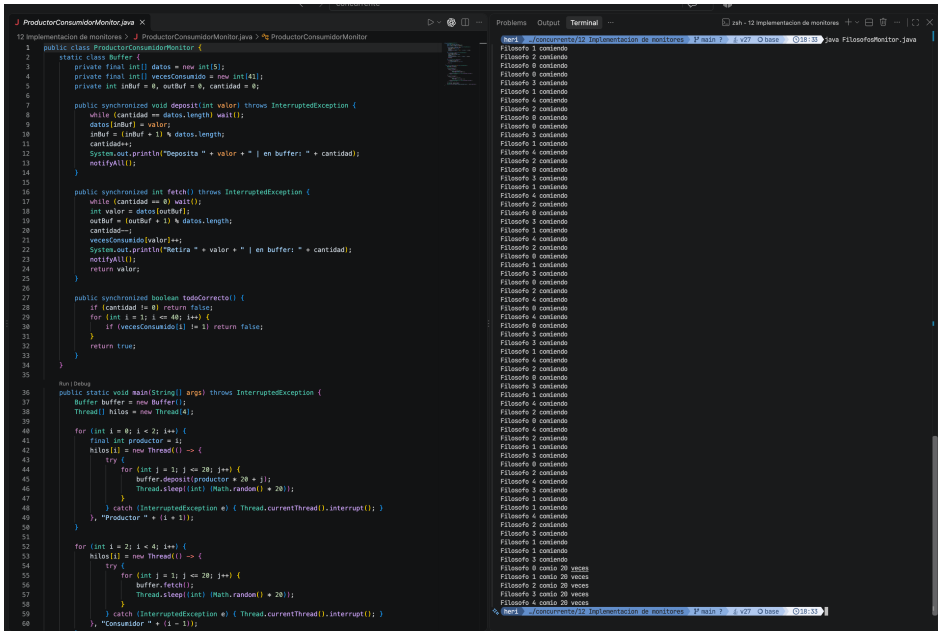
## 2.2 Prueba de escritorio

**Cuadro 2.1:** Ejemplo de dos filósofos vecinos.

| Paso | Acción | Consecuencia |
|---|---|---|
| 1 | Filósofo 0 solicita comer. | Si los vecinos no comen, pasa a COMIENDO. |
| 2 | Filósofo 1 solicita comer. | Queda HAMBRIENTO y espera porque su vecino 0 está comiendo. |
| 3 | Filósofo 0 termina. | Cambia a PENSANDO y ejecuta `test(1)`. |
| 4 | Filósofo 1 es notificado. | Recomprueba su estado y puede comer. |

## 2.3 Resultados

En diez ejecuciones con pausas aleatorias terminaron los cinco hilos. Todos registraron veinte comidas por filósofo. La exclusión entre vecinos se garantiza mediante la condición de `test(i)` dentro del monitor.



```
ProductorConsumidorMonitor.java
12 implementacion de monitores > ProductorConsumidorMonitor.java > ProductorConsumidorMonitor
public class ProductorConsumidorMonitor {
    static class Buffer {
        private final int[] datos = new int[5];
        private final int[] vecesConsumido = new int[41];
        private int inBuf = 0, outBuf = 0, cantidad = 0;

        public synchronized void deposit(int valor) throws InterruptedException {
            while (cantidad == datos.length) wait();
            datos[inBuf] = valor;
            inBuf = (inBuf + 1) % datos.length;
            cantidad++;
            System.out.println("Deposita " + valor + " | en buffer: " + cantidad);
            notifyAll();
        }

        public synchronized int fetch() throws InterruptedException {
            while (cantidad == 0) wait();
            int valor = datos[outBuf];
            outBuf = (outBuf + 1) % datos.length;
            cantidad--;
            vecesConsumido[valor]++;
            System.out.println("Retira " + valor + " | en buffer: " + cantidad);
            notifyAll();
            return valor;
        }

        public synchronized boolean todoCorrecto() {
            if (cantidad != 0) return false;
            for (int i = 1; i <= 40; i++) {
                if (vecesConsumido[i] != 1) return false;
            }
            return true;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Buffer buffer = new Buffer();
        Thread[] hilos = new Thread[4];

        for (int i = 0; i < 2; i++) {
            final int productor = i;
            hilos[i] = new Thread(() -> {
                try {
                    for (int j = 1; j <= 20; j++) {
                        buffer.deposit(productor * 20 + j);
                        Thread.sleep((int) (Math.random() * 20));
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }, "Productor " + (i + 1));
        }

        for (int i = 2; i < 4; i++) {
            hilos[i] = new Thread(() -> {
                try {
                    for (int j = 1; j <= 20; j++) {
                        buffer.fetch();
                        Thread.sleep((int) (Math.random() * 20));
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            }, "Consumidor " + (i - 1));
```

```
Problems  Output  Terminal
zsh - 12 implementacion de monitores
her1  ~/Concurrente/12 Implementacion de monitores  main ?  v27  base  18:33  java FilosofosMonitor.java
Filosofo 1 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 0 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 0 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 0 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 1 comiendo
Filosofo 3 comiendo
Filosofo 0 comiendo
Filosofo 2 comiendo
Filosofo 4 comiendo
Filosofo 0 comiendo
Filosofo 4 comiendo
Filosofo 0 comiendo
Filosofo 3 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 0 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 1 comiendo
Filosofo 3 comiendo
Filosofo 0 comiendo
Filosofo 2 comiendo
Filosofo 4 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 1 comiendo
Filosofo 4 comiendo
Filosofo 2 comiendo
Filosofo 3 comiendo
Filosofo 1 comiendo
Filosofo 1 comiendo
Filosofo 3 comiendo
Filosofo 0 comio 20 veces
Filosofo 1 comio 20 veces
Filosofo 2 comio 20 veces
Filosofo 3 comio 20 veces
Filosofo 4 comio 20 veces
her1  ~/Concurrente/12 Implementacion de monitores  main ?  v27  base  18:33
```

**Figura 2.1:** Código y salida real de los filósofos con monitor.

# 3. Resultados comparados y análisis

**Cuadro 3.1:** Resultados del productor–consumidor en diez ejecuciones.

| Ejecución | ¿Terminó? | ¿Acceso incorrecto? | ¿Hilo detenido? |
|---|---|---|---|
| 1 | Sí | No | No |
| 2 | Sí | No | No |
| 3 | Sí | No | No |
| 4 | Sí | No | No |
| 5 | Sí | No | No |
| 6 | Sí | No | No |
| 7 | Sí | No | No |
| 8 | Sí | No | No |
| 9 | Sí | No | No |
| 10 | Sí | No | No |

En las diez pruebas, los 40 elementos se consumieron exactamente una vez.

**Cuadro 3.2:** Resultados de los filósofos en diez ejecuciones.

| Ejecución | ¿Terminó? | ¿Acceso incorrecto? | ¿Hilo detenido? |
|---|---|---|---|
| 1 | Sí | No | No |
| 2 | Sí | No | No |
| 3 | Sí | No | No |
| 4 | Sí | No | No |
| 5 | Sí | No | No |
| 6 | Sí | No | No |
| 7 | Sí | No | No |
| 8 | Sí | No | No |
| 9 | Sí | No | No |
| 10 | Sí | No | No |

En las diez pruebas, cada filósofo completó sus 20 comidas.

**1. ¿Se garantiza exclusión mutua?** Sí. En el búfer, los métodos `synchronized` impiden que dos hilos modifiquen simultáneamente los índices y la cantidad. En la mesa, el monitor actualiza los estados de forma excluyente y solo deja comer a un filósofo si ninguno de sus vecinos está COMIENDO.

**2. ¿Puede ocurrir interbloqueo?** Bajo las condiciones normales de estas implementaciones, no. Los productores esperan únicamente si faltan espacios y los consumidores únicamente si faltan elementos; las notificaciones reanudan a los bloqueados. En la mesa, los filósofos no retienen un tenedor mientras esperan otro: el monitor concede directamente el permiso de comer cuando ambos vecinos están libres.

**3. ¿Puede ocurrir inanición?** La solución del búfer completa el trabajo de estos cuatro hilos en las pruebas. Sin embargo, `notifyAll()` no garantiza un orden justo de entrada. En el monitor de filósofos, la ausencia de una cola FIFO implica que un filósofo hambriento podría esperar indefinidamente en una ejecución adversa.

**4. ¿Qué evidencia respalda las respuestas?** Los diez ensayos terminaron, los cuarenta valores fueron retirados una sola vez y cada filósofo registró veinte comidas. Las condiciones de entrada protegidas por `while` o `test(i)` explican la exclusión mutua; las pruebas no demuestran por sí solas ausencia absoluta de inanición.

**5. ¿Qué modificarían si detectaran problemas?** Para evitar inanición podría agregarse una cola FIFO de solicitudes o un turno justo. Los estados y cambios del búfer deben continuar protegidos por el monitor.

**Conclusión.** Un monitor reúne los datos compartidos y las operaciones protegidas. `synchronized` proporciona exclusión mutua; `wait()` suspende hilos cuando la condición no se cumple; y `notifyAll()` les permite volver a comprobarla. Las diez pruebas completaron la ejecución y confirmaron los resultados esperados, aunque la justicia futura requiere un mecanismo adicional.